Entrega 2: run and review guide

Compiscript produces typed, located three-address code and a static extended-symbol report through one compiler core. Lowering covers all valid AST construction kinds, including the values/effects and guarded unknown-value/try-catch additions. Start here from the repository root. It is a compiler, not a source or TAC execution service. The report separates implemented contracts from grading, authorship and manual presentation checks.

Documentation map

Need	Reference
Academic outcome, PDF requirements, contributions, resolved gaps and human checks	Technical report
Pipeline, ownership and practical design decisions	Architecture
Full instruction syntax, translation examples and backend assumptions	TAC reference
Successful/failed executable battery and evidence boundaries	Acceptance
Load/edit/compile/analyze, panels and HTTP guards	IDE guide
Complete four-page assignment, original page boundaries and rubric	Canonical instructions

Downloadable PDF packet

- Technical report
- Architecture
- Run and review guide
- Detailed TAC reference, translation examples and assumptions

The PDFs are readable exports of the editable Markdown sources above, not new implementation or browser-certification evidence. Keep the four PDFs together for packet links; other references open repository branch/source-path URLs, not immutable permalinks. See the deliverables index and version policy. The canonical instructions preserve the complete assignment; the original `Proyecto_2__Compiladores.pdf` stays local and ignored. Regenerate the packet after changing its Markdown sources and version both together.

Prerequisites

- Go 1.26.1 or later, as declared in go.mod; current local verification used Go 1.27.2 on darwin/arm64.
- Existing Go module dependencies: ANTLR Go runtime v4.13.1 and the pinned indirect `golang.org/x/exp` version in `go.mod`. No dependency change is required.
- Node.js for `node --test`; observed version 24.21.0. No npm install is needed for these built-in tests.
- Java for ANTLR regeneration only; observed OpenJDK 17.0.20.1. The generator jar is ANTLR 4.13.2, cached at `tools/antlr/antlr-4.13.2-complete.jar`. Generator and Go runtime versions are distinct. A fresh offline checkout needs the same cached jar/modules; they are not guaranteed by Markdown or by Git alone.
- A browser for the local IDE. Existing CodeMirror/Viz.js CDN assets make browser appearance and offline operation dependent on asset availability. No offline-browser guarantee is made.

Generate code

```
go run ./cmd/compiscript-tac -h
go run ./cmd/compiscript-tac testdata/compiscript/tac/numeric.cps
go run ./cmd/compiscript-tac -format=json testdata/compiscript/tac/acceptance.cps
go run ./cmd/compiscript-tac -format=json testdata/compiscript/tac/values-effects.cps
go run ./cmd/compiscript-tac -format=json testdata/compiscript/tac/dynamic-exceptions.cps
go run ./cmd/compiscript testdata/compiscript/tac/acceptance.cps
```

The numeric TAC ends in a store to `s1.x:float`; it represents `x = 3.5` but does not print that runtime value. Successful compilation JSON contains `tac`, empty `diagnostics`, semantic `scopes` and optional `codegen` with unit `abstract-cell`. The last command is the legacy Analyze interface: AST/diagnostics/scopes, no TAC or codegen additions.

The two new fixtures generate print/guard/handler instructions. Only the bounded test oracle executes them: `values-effects.cps` produces `3\n1\nnull\n` with `x=2`, `y=3`, `count=1`; `dynamic-exceptions.cps` produces `3\ndivision by zero\n11\n` with `result=3`, `effects=11`. These are test expectations, not CLI stdout. Unknown values may compile successfully and then encounter an explicit runtime fault; compilation is not execution safety certification. See the values/handler contracts.

Failed input and exact exit status

```
go run ./cmd/compiscript-tac testdata/compiscript/acceptance/operators_invalid.cps
go run ./cmd/compiscript-tac -format=json testdata/compiscript/acceptance/operators_invalid.cps
```

This fixture yields six `SEM_OPERATOR` diagnostics. Text rejection has empty stdout and located errors on stderr. JSON rejection has the report on stdout, empty stderr, `tac: null` and no `codegen`. A built TAC binary exits **0** on success/help, **1** on diagnostics or I/O failure, **2** on argument/format/extension errors. Flags precede the path. Legacy Analyze still exits **0** for language diagnostics. `go run` wraps nonzero program exits; use the existing built-CLI tests for exact status proof, not the wrapper status.

Use the IDE

```
go run ./cmd/ide
```

Open `http://localhost:8080`. The address `:8080` and relative `web` file root are defined in cmd/ide/main.go, so launch from the repository root. The server binds the wildcard address, not loopback exclusively; use only in a trusted local environment. Stop the foreground server with Ctrl+C.

Select Compiscript, load a `.cps` file or edit source, then choose **Compile .cps**. TAC, diagnostics, environments and extended symbols are separate views. **Analyze .cps (AST)** keeps the older analysis workflow. See IDE details for reset, stale responses and YAPar preservation.

For a server already started locally, this request illustrates the public contract (not an execution request):

```
curl -sS http://localhost:8080/api/compiscript/compile -H 'Content-Type: application/json' --data
'{"source":"let x=1+2;"}'
```

HTTP 200 can contain language diagnostics; it does not mean the source compiled successfully. Request failures use 400/413/415/405 separately. No listener or curl request was started during the documentation check; httptest proves the handler boundary without a lingering server.

Run the battery safely

Use the focused acceptance commands. They exercise success, atomic rejection, CLI statuses, HTTP parity and bounded execution of emitted TAC. Broad `go test ./...` can regenerate ignored ANTLR files; run full generation/build/vet checks only in an isolated source snapshot. Offline checks require `GOPROXY=off GOSUMDB=off GOTOOLCHAIN=local`, existing `GOMODCACHE`, and external HOME/TMPDIR/GOCACHE. Do not infer an offline installation capability from an already cached run.

Before presentation

Review the current disposition and human checks, manually inspect the browser, and verify GitHub access and actual individual authorship. The original accepted-construction gap is resolved by T9/T10; no additional concrete normative implementation gap is established by this refresh. That is not a proof of every program or future presentation conditions. The PDF's 25/65/10 points are grading weights, not completion percentages.